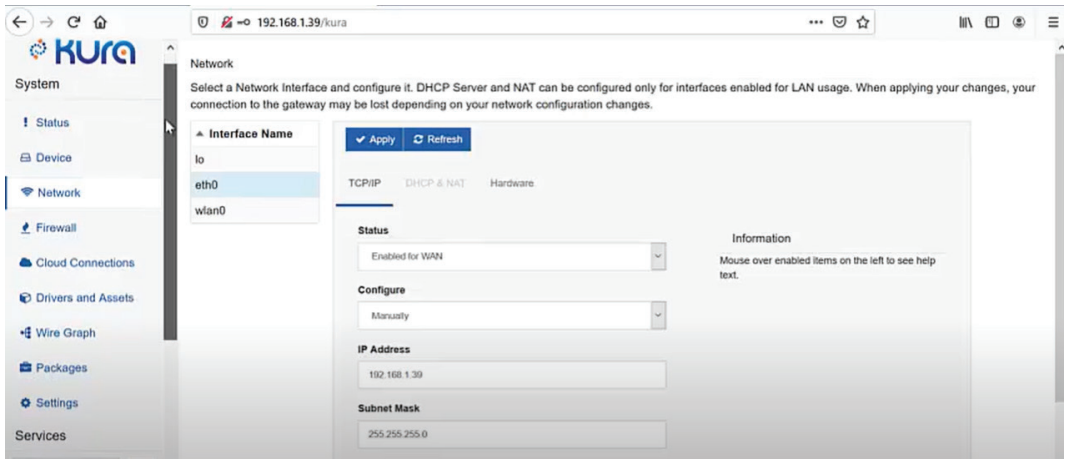

Figure 2: Kura configuration dashboard

```
public class SensorPublisher implements KuraApplication,
ConfigurableComponent {
    private DataService dataService;
    @Reference
    public void setDataService(DataService dataService) {
        this.dataService = dataService;
    }

    @Override
    public void start() {
        System.out.println("Application Started");
    }

    @Override
    public void stop() {
        System.out.println("Application Stopped");
    }

    public void publishMessage() {
        try {
            KuraPayload payload = new KuraPayloadBuilder()
                .withMetric("temperature", 25.5)
                .withMetric("humidity", 60)
                .build();

            dataService.publish("sensors/data", payload, 0,
false, 10);
            System.out.println("Message Published");
        } catch (KuraException e) {
            e.printStackTrace();
        }
    }
}
```

Packaging and deploying the application

Run *mvn* clean package to create a *.jar* file in the target directory. To upload to Kura, access the Kura web UI. Go to *Applications* > *Install/Upgrade*. Upload the *.jar* file of your application.

To run the application, start it from the Kura Web UI. Check logs to verify the application is running by going to *System* > *Log*.

Test MQTT publishing

Several steps are required to test MQTT publishing in Eclipse Kura to ensure that your system can send (publish) data to an MQTT broker successfully. Eclipse Kura offers a comprehensive platform for connecting devices to the Internet of Things (IoT), and MQTT is a popular messaging protocol for communicating between devices and servers.

- Ensure your MQTT broker is running.
- Use a client like MQTT Explorer to subscribe to the sensors/data topic.
- Verify the published data. For example:

Json code:

```
{
    "temperature": 25.5,
    "humidity": 60
}
```

Add configuration parameters for the MQTT topic, QoS, or sensor intervals using Kura's configuration service. Implement data filtering or transformation and integrate with other protocols (e.g., Modbus or OPC-UA).

The Kura framework enables easy interconnectedness of sensors and devices without the source code modifications intended to collect temperature data in real-time. Using its extensive features, including device management, connectivity, and data storage, Kura can assist in the deployment of temperature monitoring systems for smart homes, industrial automation, and environmental monitoring, among other applications. END

By: Dr R. Anand

The author is a professor in the Department of Computer Science and Engineering at Easwari Engineering College, Chennai, Tamil Nadu.